

Analisi e documentazione del progetto



Alessandro Luppino

Politecnico di Bari
Ingegneria del Software

Contents

1	Introduzione	2
2	Contesto e stato dell'arte	2
3	Requisiti	2
3.1	Funzionali	2
3.2	Non Funzionali	2
4	Modello di processo	3
5	Analisi dei Costi	3
5.1	Hosting del modello	3
5.2	Fine-tuning del modello	4
5.2.1	In locale	4
5.2.2	In Cloud	4
6	Problemi riscontrati e Scelte tecniche	4
7	Prototipo	5
8	Stato Attuale	5
9	Sviluppi Futuri	7

1 Introduzione

Il progetto consiste nello sviluppo di una chat da terminale per sistemi Linux, che consente di interagire con un modello linguistico avanzato (LLM)[8], appositamente integrato con strumenti di sicurezza esistenti.

L'obiettivo principale è l'applicazione del modello di intelligenza artificiale nell'ambito della cybersecurity, con lo scopo di assistere l'utente nella prima fase di penetration testing[1] fornendo un'esperienza interattiva.

2 Contesto e stato dell'arte

Nel panorama attuale della cybersecurity, l'intelligenza artificiale ha assunto un ruolo sempre più centrale[6][9], con modelli avanzati che supportano analisi delle minacce, rilevamento delle vulnerabilità e simulazioni di attacchi[2]. Molte soluzioni disponibili sul mercato fanno uso di modelli altamente specializzati, addestrati su dataset specifici relativi alla sicurezza informatica, che garantiscono performance ottimizzate. Tuttavia, questi strumenti presentano spesso delle limitazioni, come la necessità di accedere a servizi remoti, restrizioni nel numero di richieste effettuabili e costi associati all'uso di modelli complessi.

AI PenTesting Suite si inserisce in questo contesto differenziandosi dai tradizionali servizi AI in quanto prevede l'hosting del modello sulla macchina dell'utente (di consueto si utilizza un'architettura Client-Server), e l'integrazione con strumenti di cybersecurity affermati.

3 Requisiti

3.1 Funzionali

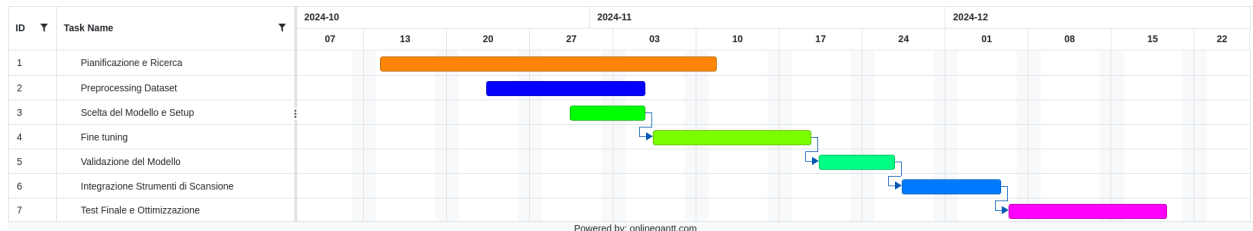
- **Interfaccia conversazionale:** il sistema deve consentire agli utenti di interagire tramite una chat da terminale.
- **Scansione delle vulnerabilità:** deve eseguire scansioni delle infrastrutture IT utilizzando strumenti come Nmap e analizzarne i risultati per identificare vulnerabilità, assegnando un livello di rischio.
- **Ricerca delle vulnerabilità:** deve consentire agli utenti di cercare informazioni su vulnerabilità specifiche, restituendo dettagli e suggerimenti.

3.2 Non Funzionali

- **Prestazioni:** il sistema deve elaborare i dati di una scansione e generare risposte in tempi ragionevoli.
- **Scalabilità:** deve essere facilmente estendibile per includere nuove integrazioni.
- **Usabilità:** l'output deve essere chiaro e diretto.
- **Affidabilità:** il sistema deve fornire risultati accurati relativi alle vulnerabilità rilevate.

4 Modello di processo

Il progetto è stato sviluppato seguendo un approccio iterativo e incrementale, integrato con una chiara definizione dei requisiti funzionali e non funzionali. Questo approccio si è rivelato efficace per affrontare lo sviluppo di un sistema complesso, che è stato suddiviso in moduli per facilitare il testing durante lo sviluppo e gli aggiornamenti futuri. L'iterazione costante ha permesso di soddisfare i requisiti specificati in modo graduale, garantendo un miglioramento continuo e un risultato stabile.



Il Gantt iniziale del progetto.

5 Analisi dei Costi

I risultati di questa analisi sono quelli che hanno determinato le scelte che verranno poi discusse.

5.1 Hosting del modello

L'hosting di un modello da 1.5B parametri su una piattaforma come Hugging Face Spaces richiede risorse significative per garantire prestazioni accettabili durante le interazioni: almeno una GPU con 16-24 GB di memoria per essere caricato e mantenere tempi di risposta adeguati.

Hugging Face Spaces offre un piano gratuito, ma con risorse limitate, non adatte a modelli di questa scala. Questo è stato verificato durante i test, dove il sistema risultava così lento da essere inutilizzabile.

Per accedere a risorse adeguate, è quindi necessario un piano premium o l'uso di servizi cloud come AWS, Google Cloud o Azure.

su Hugging Face Spaces, un'istanza GPU T4 dedicata costa circa \$0.60-\$1.00 all'ora, mentre un'istanza con una GPU A100 (più adatta per i carichi richiesti dal progetto) può salire a \$2.00-\$4.00 all'ora. Supponendo un utilizzo continuo di 24 ore al giorno in modo che il modello sia sempre disponibile, i costi variano da \$432 a \$2880 al mese a seconda delle risorse selezionate.

NOTA: Ci sono alternative più economiche di quella analizzata, piattaforme come Google Colab Pro+ o Paperspace offrono GPU a tariffe ridotte, ma la scalabilità e la disponibilità per un'applicazione a lungo termine restano problematiche e l'hosting su uno di questi servizi è troppo dispendioso.

5.2 Fine-tuning del modello

5.2.1 In locale

Il fine-tuning di un modello da 1.5B parametri è un processo computazionalmente intenso che richiede hardware e risorse software specifiche[3]:

- **GPU con almeno 24 GB di VRAM:** quantità minima necessaria per gestire il training in batch ragionevoli.
- **CPU potente:** per gestire il pre-processing dei dati e il training in parallelo.
- **Ampio spazio di archiviazione:** per caricare dataset e checkpoint intermedi.

La mia macchina non soddisfa nessuno di questi requisiti, e l'acquisto di hardware dedicato per il fine-tuning comporterebbe spese troppo elevate.

NOTA: Anche se fosse possibile suddividere i batch per adattarsi a una GPU con meno memoria, i tempi di elaborazione crescerebbero esponenzialmente, rendendo il processo impraticabile.

5.2.2 In Cloud

Un'alternativa più economica per il fine-tuning sono le stesse piattaforme Cloud discusse nell'hosting del modello 5.1. Il training completo di un modello da 1.5B parametri può richiedere sulle 48-72 ore di calcolo su una GPU V100. Un'istanza GPU V100 su AWS costa circa \$2.50-\$3.00 all'ora, mentre una GPU A100 può costare fino a \$4.00-\$5.00 all'ora. Il costo totale varia quindi da \$120 a \$360 per il fine-tuning (escludendo costi per archiviazione e trasferimento dati).

NOTA: il fine-tuning richiede anche dataset specifici e pre-elaborati, il che aggiunge tempo e possibili costi.

6 Problemi riscontrati e Scelte tecniche

Uno dei principali problemi affrontati durante lo sviluppo del progetto, è stata l'impossibilità tecnica di effettuare il fine-tuning del modello AI su dati specifici del dominio della cybersecurity[7]. Questo limite, legato principalmente a risorse computazionali insufficienti e a costi elevati, ha portato a scegliere un modello general-purpose già pre-addestrato, accettando compromessi in termini di specificità delle risposte.

Per mitigare gli effetti negativi di questa scelta e le "allucinazioni" tipiche di qualsiasi LLM, il sistema è stato, attualmente, integrato con:

- **nmap[4]:** per effettuare scansioni di reti e porte, fornendo dettagli utili per valutazioni di sicurezza.
- **nvdb[5]:** una libreria python per interagire con il NVD (National Vulnerability Database), permettendo la ricerca e il controllo di eventuali CVE (Common Vulnerabilities and Exposures) associate ai servizi.

E prevede l'integrazione di ulteriori strumenti in futuro. (vedi Sviluppi Futuri)

Queste integrazioni permettono di compensare la mancanza di conoscenza specifica del modello, migliorando l'accuratezza, l'affidabilità e la ripetibilità delle risposte.

Un altro aspetto distintivo del progetto è il prima citato hosting locale sulla macchina dell'utente, che garantisce piena autonomia e libertà di utilizzo del sistema senza restrizioni. Tuttavia, questa scelta

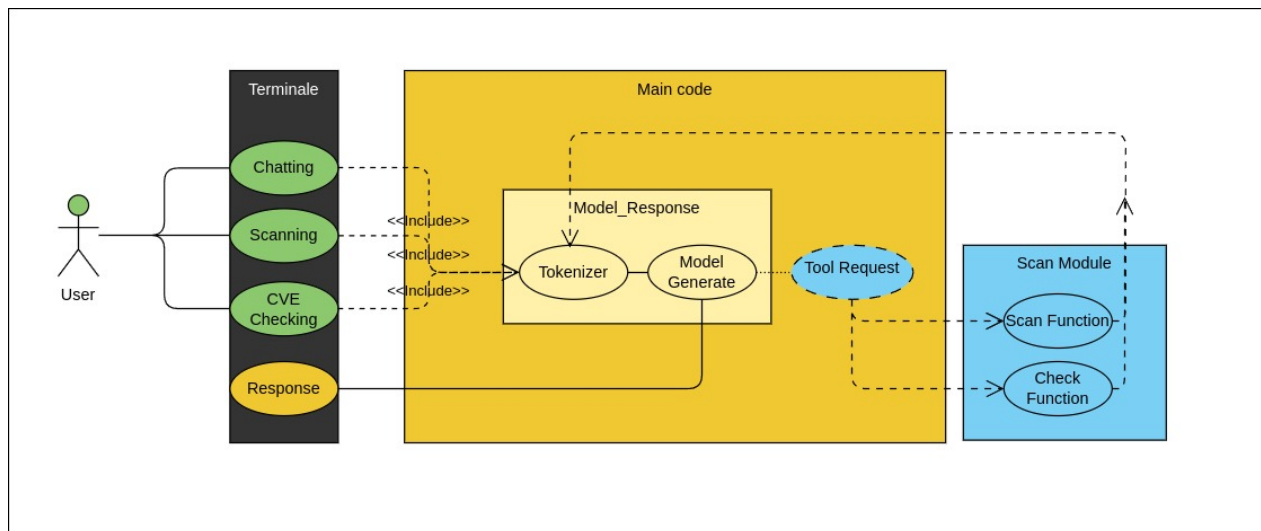
comporta un aumento del carico computazionale sulla macchina stessa, richiedendo un compromesso tra performance del sistema e risorse disponibili. La decisione di privilegiare un'architettura locale, oltre a fornire una soluzione completamente gratuita, riflette l'obiettivo di creare uno strumento flessibile, senza dipendenze da infrastrutture esterne, adatto a scenari dove la privacy e la disponibilità del modello giocano un ruolo critico.

Ad Es. è possibile conversare con il modello anche in assenza di connessione internet seppure si svanno a sacrificare alcune funzionalità.

7 Prototipo

Il primo prototipo del progetto è stato presentato in data 29/11/2024.

In questa fase, il sistema implementava le funzionalità di base, soddisfacendo parzialmente i requisiti funzionali e non funzionali precedentemente definiti. Il prototipo era già operativo e in grado di svolgere alcune funzionalità più complesse come la scansione di reti tramite Nmap. Il problema principale riscontrato è stato il tempo di risposta del modello, soprattutto durante le scansioni i tempi di attesa superavano il minuto, rendendo l'esperienza utente non ottimale.

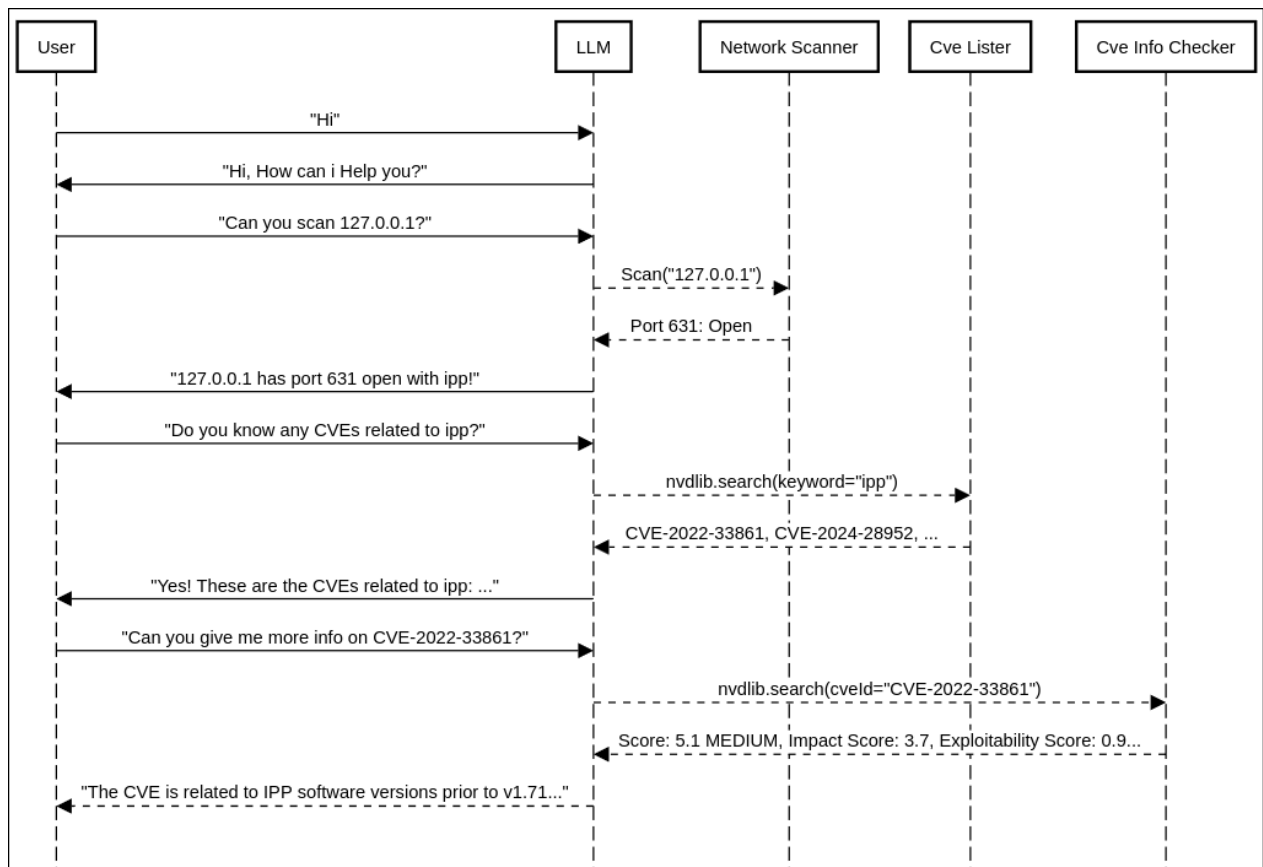
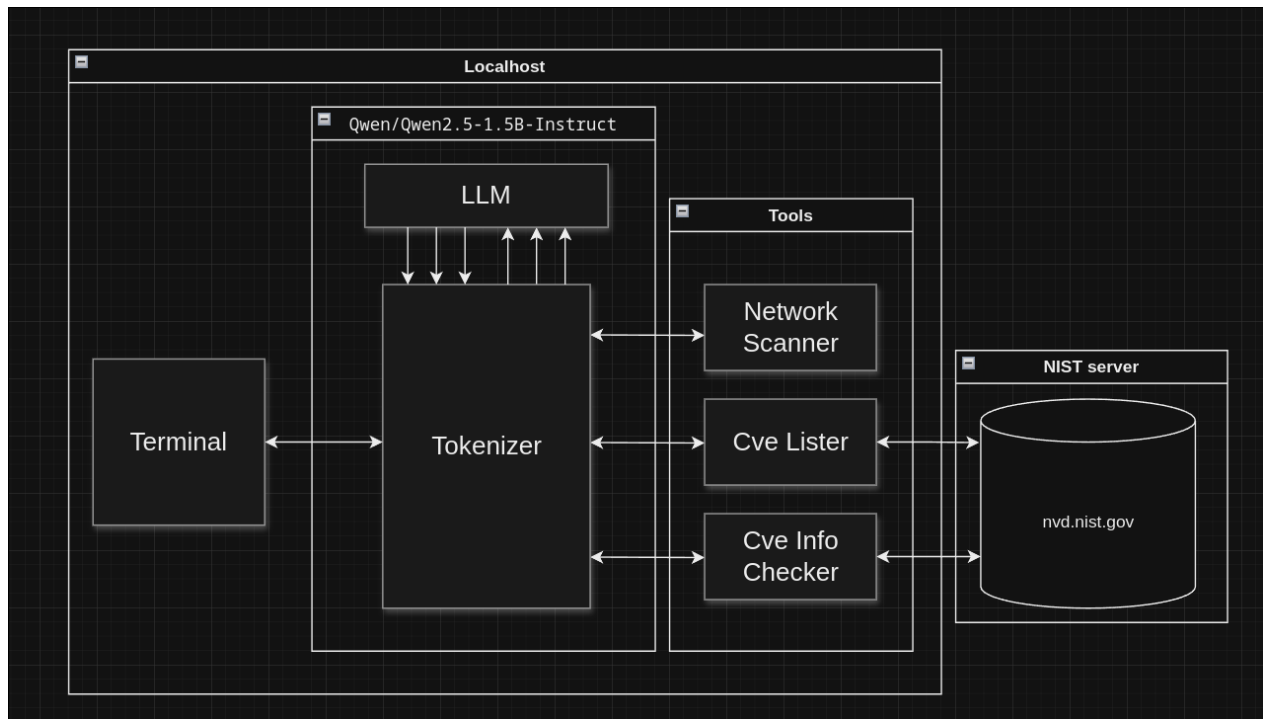


Il diagramma dei casi d'uso seguito anche dal prototipo.

8 Stato Attuale

Dopo aver analizzato attentamente i limiti del prototipo, sono state effettuate numerose migliorie all'esperienza utente, tra cui un'interfaccia grafica molto più pulita che permette al modello di formattare le risposte, l'aggiunta di un sistema di cronologia in modo che l'utente possa automaticamente ripetere comandi passati più facilmente, e la completa rivisitazione del sistema che gestiva l'input e l'output del modello, per permettere di vedere in tempo reale i messaggi del modello, riducendo notevolmente i tempi di attesa.

Oltre a questo sono state aggiunte nuove funzionalità come la ricerca di CVE tramite NVD e resa più efficace l'integrazione con nmap.



Architettura e diagramma delle sequenze della versione finale del progetto.

9 Sviluppi Futuri

Gli sviluppi futuri del progetto mirano a potenziarne ulteriormente le funzionalità e migliorare l'esperienza utente. È previsto l'aggiungimento di nuovi moduli, come un Modulo di Attacco, che suggerirà exploit e tecniche di attacco specifiche per le vulnerabilità rilevate, e un Modulo di Report, capace di generare report dettagliati e professionali utili per la documentazione del lavoro svolto. Inoltre, si intende implementare un sistema per salvare le conversazioni e consentire agli utenti di riprenderle in futuro, garantendo continuità nelle analisi.

Sul fronte dell'usabilità, lo sviluppo di un'interfaccia grafica più intuitiva rappresenta una priorità, insieme a funzionalità che migliorino l'interazione, come la possibilità di interrompere il modello durante la generazione di output non soddisfacenti.

Infine, se i fondi lo permetteranno, si prevede di effettuare il fine-tuning del modello AI. Questo consentirebbe di adattare il sistema a scenari specifici della cybersecurity, migliorandone la precisione e riducendo ulteriormente il rischio di allucinazioni.

Stack Tecnico

Il modello utilizzato per il progetto è **Qwen2.5-1.5B-Instruct**, un modello AI general-purpose con 1.5 miliardi di parametri, scelto per le sue capacità avanzate di generazione e comprensione del linguaggio naturale. Lo sviluppo è stato effettuato in Python, versione 3.12.8, con il supporto delle seguenti librerie:

- **json, time, os, re, socket**: per la gestione di operazioni di base del sistema e manipolazione dei dati.
- **nvdlib, nmap, whois**: per l'integrazione con strumenti e librerie di cybersecurity.
- **prompt_toolkit** (moduli: `PromptSession`, `HTML`, `KeyBindings`, `FileHistory`, `run_in_terminal`, `AutoSuggestFromHistory`): per implementare l'interfaccia conversazionale interattiva.
- **rich** (moduli: `Console`, `Live`, `Markdown`, `Spinner`): per fornire un output visuale avanzato e leggibile.
- **threading.Thread**: per gestire l'esecuzione di task concorrenti.
- **transformers** (moduli: `AutoTokenizer`, `AutoModelForCausalLM`, `TextIteratorStreamer`): per il caricamento e l'interazione con il modello AI.
- **torch** (*PyTorch*): come backend per il machine learning, necessario per il funzionamento del modello.

Test e sviluppo sono stati eseguiti su una macchina con AMD Ryzen 5 7535U e 16 GB di RAM.

References

- [1] Mariam Alhamed and M. M. Hafizur Rahman. “A Systematic Literature Review on Penetration Testing in Networks: Future Research Directions”. In: *Applied Sciences* 13.12 (2023). ISSN: 2076-3417. DOI: 10.3390/app13126986. URL: <https://www.mdpi.com/2076-3417/13/12/6986>.
- [2] Andreas Happe and Jürgen Cito. “Getting pwn’d by AI: Penetration Testing with Large Language Models”. In: ESEC/FSE 2023. San Francisco, CA, USA: Association for Computing Machinery, 2023, pp. 2082–2086. ISBN: 9798400703270. DOI: 10.1145/3611643.3613083. URL: <https://doi.org/10.1145/3611643.3613083>.
- [3] Mathav Raj J et al. *Fine Tuning LLM for Enterprise: Practical Guidelines and Recommendations*. 2024. arXiv: 2404.10779 [cs.SE]. URL: <https://arxiv.org/abs/2404.10779>.
- [4] *Nmap Documentation*. URL: <https://nmap.org/docs.html>.
- [5] *Nvdlb Documentation*. URL: <https://nvdlbdocs.readthedocs.io/en/latest/>.
- [6] Oluebube Princess Egbuna. “The Impact of AI on Cybersecurity: Emerging Threats and Solutions”. In: *Journal of Science Technology* 2.2 (2021), pp. 43–67. URL: <https://nucleuscorp.org/jst/article/view/232>.
- [7] University of Trento - Italy. *Datasets on Security Research at the University of Trento*. 2021. URL: <https://securitylab.disi.unitn.it/doku.php?id=datasets>.
- [8] Yifan Yao et al. “A survey on large language model (LLM) security and privacy: The Good, The Bad, and The Ugly”. In: *High-Confidence Computing* 4.2 (2024). ISSN: 2667-2952. DOI: <https://doi.org/10.1016/j.hcc.2024.100211>. URL: <https://www.sciencedirect.com/science/article/pii/S266729522400014X>.
- [9] Jie Zhang et al. “When LLMs Meet Cybersecurity: A Systematic Literature Review”. In: (2024). arXiv: 2405.03644 [cs.CR]. URL: <https://arxiv.org/abs/2405.03644>.